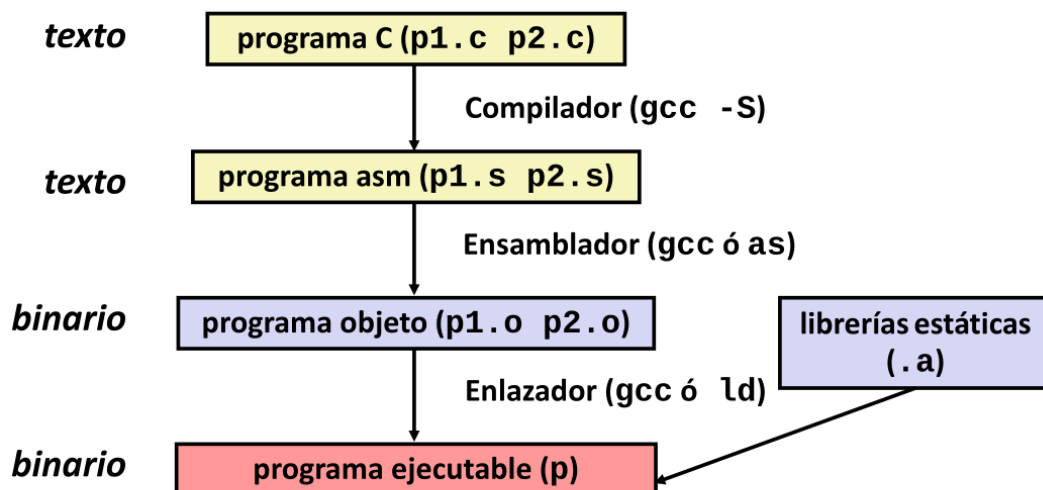


Convertir C en Código Objeto

- Código en ficheros **p1.c p2.c**
- Compilar con el comando: **gcc -Og p1.c p2.c -o p**
 - Usar optimizaciones básicas (**-Og**) [versiones recientes de GCC[†]]
 - Poner binario resultante en fichero **p**



Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition [†] GCC >= 4.8. Como referencia, Ubuntu 18.04 LTS viene con GCC 7.3 15

Tamaños

Representación Datos C, IA32, x86-64

Tamaño de Objetos C (en Bytes)

Tipo de Datos C	Normal 32-bit	Intel IA32	x86-64
▪ unsigned	4	4	4
▪ int	4	4	4
▪ long int	4	4	8
▪ char	1	1	1
▪ short	2	2	2
▪ float	4	4	4
▪ double	8	8	8
▪ long double	8	10/12	16
▪ char *	4	4	8

– o cualquier otro puntero

Registros enteros x86-64

%rax	%eax	%ax	%al	%r8	%r8d	%r8w	%r8b
%rbx	%ebx			%r9	%r9d		
%rcx	%ecx			%r10	%r10d		
%rdx	%edx			%r11	%r11d		
%rsi	%esi	%si	%sil	%r12	%r12d		
%rdi	%edi			%r13	%r13d		
%rsp	%esp			%r14	%r14d		
%rbp	%ebp			%r15	%r15d		

- Pueden referenciarse los 4 bytes de menor peso[†] (los 4 LSBs)
 - (también los 2 LSB y el 1 LSB)

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

[†] "low-order 4 bytes" 27

■ Instrucciones de Dos Operandos:

Formato

Operación[†]

addq	Src, Dest	Dest = Dest + Src
subq	Src, Dest	Dest = Dest – Src
imulq	Src, Dest	Dest = Dest * Src
salq	Src, Dest	Dest = Dest << Src
sarq	Src, Dest	Dest = Dest >> Src
shrq	Src, Dest	Dest = Dest >> Src
xorq	Src, Dest	Dest = Dest ^ Src
andq	Src, Dest	Dest = Dest & Src
orq	Src, Dest	Dest = Dest Src

También llamada shlq

Aritméticas

Lógicas

Saltos

Instrucciones de salto condicional en comparaciones		
Condición	Números sin signo	Números con signo
=	je	
≠	jne	
<	jb, jnae	j1, jnge
≥	jae, jnb	jge, jnl
>	ja, jnbe	jg, jnle
≤	jbe, jna	jle, jng
Instrucción de salto incondicional		
saltar siempre	jmp	

Ajustes condicionales

Instrucciones de salto condicional en comparaciones		
Condición	Números sin signo	Números con signo
=	sete	
≠	setne	
<	setb, setnae	setl, setnge
≥	setae, setnb	setge, setnl
>	seta, setnbe	setg, setnle
≤	setbe, setna	setle, setng

Bits de estado

- **CF** (*Carry Flag*): 1 si resultado fuera de rango sin signo
 - Suma: es una copia del acarreo en el bit más significativo
 - Resta: es la negación del acarreo en el bit más significativo
 - **PF** (*Parity Flag*): 1 si núm. unos en LSB de resultado es par
 - **ZF** (*Zero Flag*): 1 si resultado = 0
 - **SF** (*Sign flag*): 1 si resultado < 0 en operaciones con signo
 - Es una copia del bit más significativo del resultado
 - **OF** (*Overflow flag*): 1 si resultado fuera de rango con signo
-
- mov nunca afecta a los indicadores de estado
 - inc y dec no afectan a CF

Complemento a dos

- Llamamos a la representación de los negativos “representación en complemento a 2” o C_2 :
 - Para $x < 0$:

$$C_2(|x|) = 2^n - |x|$$

- Realmente usamos aritmética modular
 - Para cualquier x :

$$C_2(x) = (2^n - x) \bmod 2^n$$

■ Resumidamente:

COND	Números sin signo		Números con signo	
	Instrucción	Condición	Instrucción	Condición
=	je=jz sete=setz	ZF	je=jz sete=setz	ZF
≠	jne=jnz setne=setnz	~ZF	jne=jnz setne=setnz	~ZF
<	jb=jnae=jc setb=setnae=setc	CF	jle=jnge setl=setnge	SF^OF
≥	jae=jnb=jnc setae=setnb=setnc	~CF	jge=jnl setge=setnl	~(SF^OF)
>	ja=jnbe seta=setnbe	~CF · ~ZF ~(CF ZF)	jg=jnle setg=setnle	~(SF^OF) · ~ZF ~((SF^OF) ZF)
≤	jbe=jna setbe=setna	CF ZF	jle=jng setle=setng	(SF^OF) ZF

~ZF => ZF = 0

Uso de Registros en Linux x86-64[†] #1

■ %rax

- Valor de retorno
- También salva-invocante
- Puede ser modificado[†] por el proc.

■ %rdi, ..., %r9

- Argumentos[†]
- También salva-invocante
- Pueden ser modificados[†] por proc.

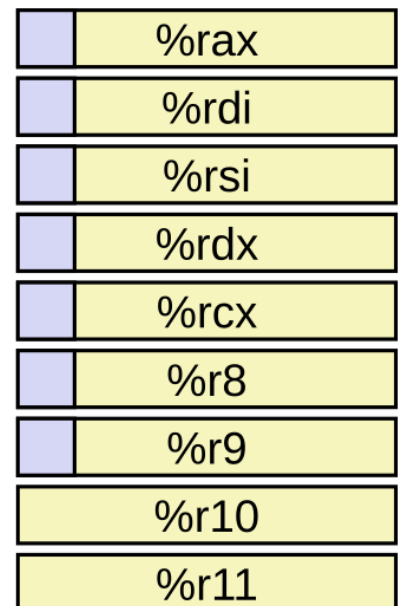
■ %r10, %r11

- Salva-invocante
- Pueden ser modificados[†] por proc.

Val.retorno
S-Invocante

Argumentos
S-Invocante

Temporales
S-Invocante



Uso de Registros en Linux x86-64 #2

■ %rbx, %r12 - %r15

- Salva-invocado
- Invocado debe preservar y restaurar

■ %rbp

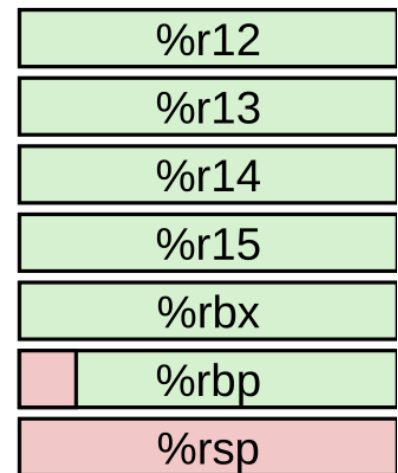
- Salva-invocado
- Invocado debe preservar y restaurar
- Puede que se use como marco pila
- Puede usarse intermezcladamente⁺

■ %rsp

- Forma especial de salva-invocado
- Restaurado a su valor original a la salida del procedimiento

Temporales
S-Invocado

Especiales



Salva-invocante: siquiero guardar un dato en uno de estos tengo llamar a la pila antes de la llamada. Esto se debe a que tienen permiso para sobrescribirlos sin avisar y al volver a la llamada el valor podría haber desaparecido.

Salva-invocado: si quieres usar estos registros debes guardar el valor anterior en la pila y restaurarlo antes de terminar, esto se debe a que la función que te llamó confía en que al volver esos registros tendrán el mismo valor que antes de llamarte.

Con estos registros hay que hacer push antes de utilizarlos, por ej pushq %rdx y popq %rdx

Casos Concretos de Alineamiento

	<i>Linux x86</i>		<i>x86-64</i>		<i>Windows MinGW32</i>		<i>MinGW64</i>	
▪ <i>Tipo de Datos C</i>	<i>tam.alin.</i>		<i>tam.alin.</i>		<i>tam.alin.</i>		<i>tam.alin.</i>	
▪ char	1	1	1	1	1	1	1	1
▪ short	2	2	2	2	2	2	2	2
▪ int	4	4	4	4	4	4	4	4
▪ long	4	4	8	8	4	4	4	4
▪ long long	8	4	8	8	8	8	8	8
▪ float	4	4	4	4	4	4	4	4
▪ double	8	4	8	8	8	8	8	8
▪ long double	12	4	16	16	12	4	16	16
▪ void *	4	4	8	8	4	4	8	8
▪ <i>Regla para memorizar</i>	4x		KB Kx		8x		KB Kx	
	ahorro M.		alin.estricto		sin ahorro		sin long 8B	